Customer Behavior Analysis

Zeshaan Sheikh

+971 567300374/ +353 833335294

suavetate@gmail.com

LinkedIN

1. Project Overview

This project analyses customer shopping behaviour using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

2. Dataset Summary

i. Rows: 3,900
ii. Columns: 18
iii. Key Features: - Customer demographics (Age, Gender, Location, Subscription Status)
iv. Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
v. Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
vi. Missing Data: 37 values in Review Rating column.

3. Exploratory Data Analysis using Python

We begin with data preparation and cleaning in Python:

i. Data Loading: Imported the dataset using pandas.
ii. Initial Exploration: Used df.info() to check structure and .describe() for summary statistics.

[4]:
```
# Summary statistics using .describe()
df.describe(include='all')
```

[4]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN

```
[5]: # Checking if missing data or null values are present in the dataset

     df.isnull().sum()

[5]: Customer ID               0
     Age                       0
     Gender                    0
     Item Purchased            0
     Category                  0
     Purchase Amount (USD)     0
     Location                  0
     Size                      0
     Color                     0
     Season                    0
     Review Rating            37
     Subscription Status       0
     Shipping Type             0
     Discount Applied          0
     Promo Code Used           0
     Previous Purchases        0
     Payment Method            0
     Frequency of Purchases    0
     dtype: int64
```

iii. Missing Data Handling: Checked for null values and imputed missing values in the Review Rating column using the median rating of each product category.

```
[6]: # Imputing missing values in Review Rating column with the median rating of the product category

     df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))

[7]: df.isnull().sum()

[7]: Customer ID               0
     Age                       0
     Gender                    0
     Item Purchased            0
     Category                  0
     Purchase Amount (USD)     0
     Location                  0
     Size                      0
     Color                     0
     Season                    0
     Review Rating             0
     Subscription Status       0
     Shipping Type             0
     Discount Applied          0
     Promo Code Used           0
     Previous Purchases        0
     Payment Method            0
     Frequency of Purchases    0
     dtype: int64
```

iv. Column Standardization: Renamed columns to snake case for better readability and documentation.

```
[11]: df[['age','age_group']].head(10)
```

[11]:

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult

v. Feature Engineering:

- Created age_group column by binning customer ages.
- Created purchase_frequency_days column from purchase data.

```
[12]: # create new column purchase_frequency_days

frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

```
[21]: df[['purchase_frequency_days','frequency_of_purchases']].head(10)
```

[21]:

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

```
[22]: df[['discount_applied','promo_code_used']].head(10)
```

[22]:

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[23]: (df['discount_applied'] == df['promo_code_used']).all()
```

```
[23]: True
```

```
[24]: # Dropping promo code used column

df = df.drop('promo_code_used', axis=1)
```

```
[25]: df.columns
```

```
[25]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='object')
```

vi. Data Consistency Check: Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

vii. Database Integration: Connected Python script to PostgreSQL and loaded the cleaned DataFrame into the database for SQL analysis.

Connecting Python script to PostgreSQL

[26]:
```
!pip install psycopg2-binary sqlalchemy
```

```
Requirement already satisfied: psycopg2-binary in c:\users\kiit\anaconda3\lib\site-packages (2.9.10)
Requirement already satisfied: sqlalchemy in c:\users\kiit\anaconda3\lib\site-packages (1.4.22)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\kiit\anaconda3\lib\site-packages (from sqlalchemy) (1.1.1)
Note: you may need to restart the kernel to use updated packages.
```

[28]:
```python
from sqlalchemy import create_engine

# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details
username = "postgres"      # default user
password = "amlan123" # the password you set during installation
host = "localhost"         # if running locally
port = "5432"              # default PostgreSQL port
database = "customer_behavior"    # the database you created in pgAdmin

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

# Step 2: Load DataFrame into PostgreSQL
table_name = "customer"   # choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")
```

```
Data successfully loaded into table 'customer' in database 'customer_behavior'.
```

4. Data Analysis using SQL (Business Transactions)

Performed structured analysis in PostgreSQL to answer key business questions

```sql
--Q1. What is the total revenue generated by male vs. female customers?
select gender, SUM(purchase_amount) as revenue
from customer
group by gender
```

	gender text	revenue numeric
1	Female	75191
2	Male	157890

ANSWER: Clearly Male customers generated more revenue.

```sql
--Q2. Which customers used a discount but still spent more than the average purchase amount?
select customer_id, purchase_amount
from customer
where discount_applied = 'Yes' and purchase_amount >= (select AVG(purchase_amount) from customer)
```

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90

Total rows: 839 | Query complete 00:00:00.122

ANSWER: This gives us the output of all customers via customer_id, total of 839 rows with their purchase amount listed above the average purchase amount.

```
-- Q3. Which are the top 5 products with the highest average review rating?
select item_purchased, round(avg(review_rating::numeric),2) as "Average Product Rating"
from customer
group by item_purchased
order by avg(review_rating) desc
limit 5
```

	item_purchased text	Average Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78

ANSWER: Gloves, Sandals, Boots, Hat & Skirt had the highest average review rating

```
--Q4. Compare the average Purchase Amounts between Standard and Express Shipping.
select shipping_type,
ROUND(AVG(purchase_amount),2)
from customer
where shipping_type in ('Standard','Express')
group by shipping_type;
```

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

ANSWER: Express shipping gives a higher purchase amount figure.

```sql
--Q5. Do subscribed customers spend more? Compare average spend and total revenue
--between subscribers and non-subscribers.
SELECT subscription_status,
       COUNT(customer_id) AS total_customers,
       ROUND(AVG(purchase_amount),2) AS avg_spend,
       ROUND(SUM(purchase_amount),2) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue,avg_spend DESC;
```

	subscription_status text	total_customers bigint	avg_spend numeric	total_revenue numeric
1	Yes	1053	59.49	62645.00
2	No	2847	59.87	170436.00

ANSWER: Non-subscribers show higher revenue on an average.

```sql
--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT item_purchased,
       ROUND(100.0 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)/COUNT(*),2) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.66
3	Coat	49.07
4	Sweater	48.17
5	Pants	47.37

ANSWER: Hat, Sneakers, Coat, Sweater and Pants show highest % of purchases after discount being applied.

```sql
--Q7. Segment customers into New, Returning, and Loyal based on their total
-- number of previous purchases, and show the count of each segment.
with customer_type as (
SELECT customer_id, previous_purchases,
CASE
    WHEN previous_purchases = 1 THEN 'New'
    WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
    ELSE 'Loyal'
    END AS customer_segment
FROM customer)

select customer_segment,count(*) AS "Number of Customers"
from customer_type
group by customer_segment;
```

	customer_segment text	Number of Customers bigint
1	Loyal	3116
2	New	83
3	Returning	701

ANSWER: 3116 Loyal customers, 83 new customers and 701 returning customers.

```sql
--Q8. What are the top 3 most purchased products within each category?
WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank
    FROM customer
    GROUP BY category, item_purchased
)
SELECT item_rank,category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <=3;
```

	item_rank bigint	category text	item_purchased text	total_orders bigint
1	1	Accessori...	Jewelry	171
2	2	Accessori...	Sunglasses	161
3	3	Accessori...	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

ANSWER:

- Accessories- jewellery, sunglasses and belt
- Clothing: Blouse, Pants, Shirt
- Footwear: Shoes, Sandals and Sneakers
- Outwear: Jacket and Coat

```sql
--Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

	subscription_status text	repeat_buyers bigint
1	No	2518
2	Yes	958

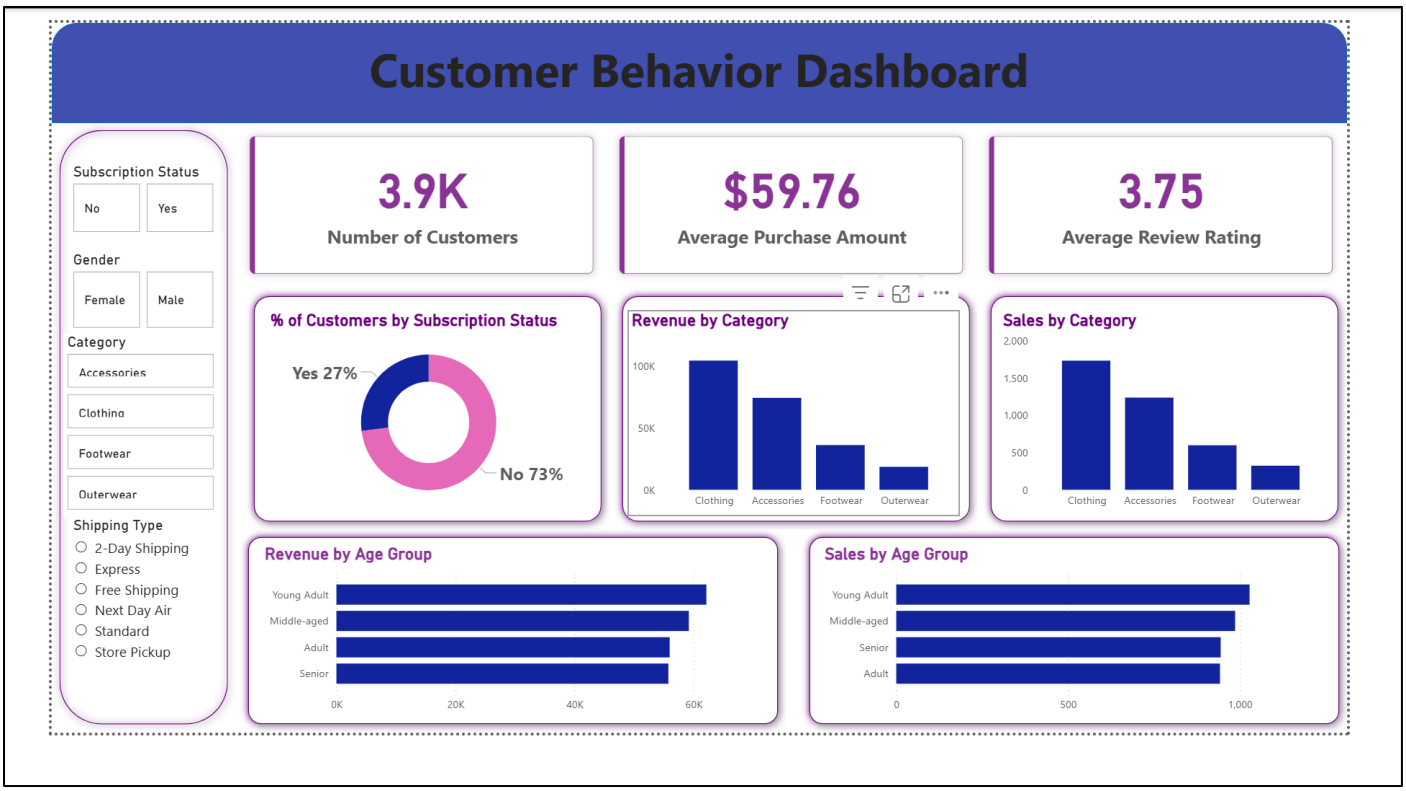
ANSWER: Non-subscribers- 2518 and subscibers-958

```
--Q10. What is the revenue contribution of each age group?
SELECT
    age_group,
    SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY age_group
ORDER BY total_revenue desc;
```

	age_group text	total_revenue numeric
1	Young Adult	62143
2	Middle-aged	59197
3	Adult	55978
4	Senior	55763

ANSWER: Young adult category shows highest purchase contribution compared to others.

5. Power BI dashboard



6. Business recommendations:

1. Boost Subscriptions – Promote exclusive benefits for subscribers.
2. Customer Loyalty Programs – Reward repeat buyers to move them into the “Loyal” segment.
3. Review Discount Policy – Balance sales boosts with margin control.
4. Product Positioning – Highlight top-rated and best-selling products in campaigns.
5. Targeted Marketing – Focus efforts on high-revenue age groups and express-shipping